

PRACTICAL-11

Task-1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

/* Thread function */
void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
    }

    printf("Thread %d completed.\n", thread_id);

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    printf("Multithreaded Counter Application\n");
    printf("-----\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Iterations/thread : %d\n", ITERATIONS);
    printf("Expected counter : %d\n\n",
          NUM_THREADS * ITERATIONS);

    /* Create threads */
```

```

GNU nano 7.2                                                                    prac11t
/* Create threads */
for (int i = 0; i < NUM_THREADS; i++) {
    thread_ids[i] = i + 1;

    if (pthread_create(&threads[i], NULL,
                      increment_counter,
                      &thread_ids[i]) != 0) {
        perror("pthread_create");
        return 1;
    }
}

/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++) {
    if (pthread_join(threads[i], NULL) != 0) {
        perror("pthread_join");
        return 1;
    }
}

printf("\nAll threads completed.\n");
printf("Final counter value: %d\n", counter);
printf("Expected value      : %d\n",
       NUM_THREADS * ITERATIONS);

if (counter != NUM_THREADS * ITERATIONS) {
    printf("\nRace condition detected!\n");
    printf("Some increments were lost because multiple threads\n");
    printf("accessed the shared counter concurrently.\n");
} else {
    printf("\nNo difference observed in this execution.\n");
    printf("Race conditions are timing-dependent.\n");
}

return 0;
}

```

```

ruhi0404@Ruhitha:~$ cd Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano prac11task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc -Wall -Wextra -pthread prac11task1.c -o prac11task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac11task1
Multithreaded Counter Application
-----
Number of threads : 4
Iterations/thread : 100000
Expected counter  : 400000

Thread 1 completed.
Thread 2 completed.
Thread 4 completed.
Thread 3 completed.

All threads completed.
Final counter value: 191556
Expected value      : 400000

Race condition detected!
Some increments were lost because multiple threads
accessed the shared counter concurrently.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

```

Some increments were lost because multiple threads
accessed the shared counter concurrently.
ruhi0404@Ruhitha:~/Operating_Systems$ ./pract1task1
Multithreaded Counter Application
-----
Number of threads : 4
Iterations/thread : 100000
Expected counter  : 400000

Thread 2 completed.
Thread 3 completed.
Thread 1 completed.
Thread 4 completed.

All threads completed.
Final counter value: 181903
Expected value      : 400000

Race condition detected!
Some increments were lost because multiple threads
accessed the shared counter concurrently.
ruhi0404@Ruhitha:~/Operating_Systems$ ./pract1task1
Multithreaded Counter Application
-----
Number of threads : 4
Iterations/thread : 100000
Expected counter  : 400000

Thread 1 completed.
Thread 2 completed.
Thread 4 completed.
Thread 3 completed.

All threads completed.
Final counter value: 189773
Expected value      : 400000

Race condition detected!
Some increments were lost because multiple threads
accessed the shared counter concurrently.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation:

The multithreaded counter application was successfully implemented using POSIX threads. Four threads were created using `pthread_create()`, and each thread performed multiple increments on the shared counter variable. The main thread used `pthread_join()` to wait for all worker threads to complete. Since all threads accessed and modified the shared counter concurrently without synchronization, a race condition occurred. The expected counter value was 400000, but the actual value could be lower because multiple threads could read and update the counter simultaneously, causing some increments to be lost. Repeated executions produced different results due to differences in thread scheduling.

Task-2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

/* Mutex for protecting shared counter */
pthread_mutex_t counter_mutex;

/* Thread function */
void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        /* Lock before accessing shared variable */
        pthread_mutex_lock(&counter_mutex);

        counter++;

        /* Unlock after updating shared variable */
        pthread_mutex_unlock(&counter_mutex);
    }

    printf("Thread %d completed.\n", thread_id);

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Iterations/thread : %d\n", ITERATIONS);
    printf("Expected counter : %d\n\n",
           NUM_THREADS * ITERATIONS);

    /* Initialize mutex */
    if (pthread_mutex_init(&counter_mutex, NULL) != 0) {
        printf("Mutex initialization failed.\n");
        return 1;
    }

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i + 1;

        if (pthread_create(&threads[i], NULL,
                          increment_counter,
                          &thread_ids[i]) != 0) {
            printf("Error creating thread %d.\n", i + 1);
            pthread_mutex_destroy(&counter_mutex);
            return 1;
        }
    }

    /* Wait for all threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_join(threads[i], NULL) != 0) {
            printf("Error joining thread %d.\n", i + 1);
            pthread_mutex_destroy(&counter_mutex);
            return 1;
        }
    }

    printf("\nAll threads completed.\n");
}
```

^G Help ^O Write Out ^W Where Is ^K Cut
 ^X Exit ^R Read File ^\ Replace ^U Past

```

}

/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++) {
    if (pthread_join(threads[i], NULL) != 0) {
        printf("Error joining thread %d.\n", i + 1);
        pthread_mutex_destroy(&counter_mutex);
        return 1;
    }
}

printf("\nAll threads completed.\n");
printf("Final counter value: %d\n", counter);
printf("Expected value      : %d\n",
        NUM_THREADS * ITERATIONS);

if (counter == NUM_THREADS * ITERATIONS) {
    printf("\nSynchronization successful!\n");
    printf("No lost updates occurred.\n");
} else {
    printf("\nUnexpected result.\n");
}

/* Destroy mutex */
pthread_mutex_destroy(&counter_mutex);

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ ./prac11task2
Multithreaded Counter with Mutex
-----
Number of threads : 4
Iterations/thread : 100000
Expected counter  : 400000

Thread 3 completed.
Thread 1 completed.
Thread 2 completed.
Thread 4 completed.

All threads completed.
Final counter value: 400000
Expected value      : 400000

Synchronization successful!
No lost updates occurred.
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac11task2
Multithreaded Counter with Mutex
-----
Number of threads : 4
Iterations/thread : 100000
Expected counter  : 400000

Thread 1 completed.
Thread 3 completed.
Thread 2 completed.
Thread 4 completed.

All threads completed.
Final counter value: 400000
Expected value      : 400000

Synchronization successful!
No lost updates occurred.
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac11task2
Multithreaded Counter with Mutex
-----
Number of threads : 4
Iterations/thread : 100000

```

Observation:

The previous race-condition counter program was modified by introducing a POSIX mutex. Four threads were created using `pthread_create()`, and each thread performed 100000 increments on the shared counter. Before accessing the shared counter, each thread acquired the mutex using `pthread_mutex_lock()`, and after completing the update, it released the mutex using `pthread_mutex_unlock()`. Unlike the race-condition version, the final counter value consistently matched the expected value of 400000. The mutex prevented multiple threads from modifying the shared variable simultaneously and eliminated lost updates. Synchronization introduced a small execution overhead because threads had to acquire and release the mutex around the critical section.